

tf.compat.v1.mixed_precision.DynamicLossScale

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/mixed_precision/DynamicLossScale

Loss scale that dynamically adjusts itself.

Function Signatures

```
tf.compat.v1.mixed_precision.DynamicLossScale(  
    initial_loss_scale=(2 ** 15), increment_period=2000,  
    multiplier=2.0 )
```

Code Examples

```
tf.compat.v1.mixed_precision.DynamicLossScale(  
    initial_loss_scale=(2 ** 15), increment_period=2000, multiplier=2.0  
)
```

```
tf.compat.v1.mixed_precision.DynamicLossScale(  
    initial_loss_scale=(2 ** 15), increment_period=2000, multiplier=2.0  
)
```

```
@classmethod  
from_config(  
    config  
)
```

```
@classmethod
```

```
from_config(  
    config  
)
```

```
get_config()
```

```
get_config()
```

```
update(  
    grads  
)
```

Documentation

Loss scale that dynamically adjusts itself.

Inherits From: `LossScale`

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.mixed_precision.experimental.DynamicLossScale`,
`tf.compat.v1.train.experimental.DynamicLossScale`

Dynamic loss scaling works by adjusting the loss scale as training progresses. The goal is to keep the loss scale as high as possible without overflowing the gradients. As long as the gradients do not overflow, raising the loss scale never hurts.

The algorithm starts by setting the loss scale to an initial value. Every N steps that the gradients are finite, the loss scale is increased by some factor. However, if a NaN or Inf gradient is found, the gradients for that step are not applied, and the loss scale is decreased by the factor. This process tends to keep the loss scale as high as possible without gradients overflowing.

Attributes

Methods

from_config

[View source](#)

Creates the LossScale from its config.

get_config

[View source](#)

Returns the config of this loss scale.

update

[View source](#)

Updates loss scale based on if gradients are finite in current step.

__call__

[View source](#)

Returns the current loss scale as a scalar float32 tensor.